Learning Heuristics for the Gapped Longest Common Subsequence Problem

Marko Djukanović[1,5], Christian Blum[2], Aleksandar Kartelj[3], Sašo Džeroski[4], and Žiga Zebec[5]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iia.csic.es
[3] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[4] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[5] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

Abstract. TODO

1 Introduction

The *Longest Common Subsequence Problem* (LCSP) [10] is a well-known combinatorial optimization problem with numerous applications in computational biology, particularly in the analysis of molecular sequences. Given an arbitrary set of input sequences $S = \{s_1, \ldots, s_m\}$ over an alphabet Σ, the goal is to determine the longest possible subsequence common to all $s_i \in S$. In recent decades, many practically motivated variants of the LCSP have been proposed by considering additional structural or biological constraints such as the constrained, arc-preserving, repetition-free version [7, 9], among others. In this work we study the *Variable Gapped* LCSP (VGLCSP), introduced in [13] and theoretically examined in [2], and experimentally in [1]. This problem extends the standard LCSP by imposing flexible distance limits ("gaps") between consecutive symbols in the resulting subsequence, allowing these gaps to vary along the input sequences.

Formally, for each input sequence $s_i \in S$, a gap function $G_{s_i} : \{1, \ldots, |s_i|\} \mapsto \mathbb{N}$ defines the maximum distance allowed between the positions of appearances of consecutive characters for resulting common subsequence in s_i. That is, if the characters of a subsequence s appear at the positions $i_i^1 < \ldots < i_i^{|s|}$ in s_i, the gap constraint is satisfied iff $i_i^x - i_i^{x-1} \leq G_{s_i}[i_i^x] + 1$ for each $x = 2, \ldots, |s|$. A common subsequence s is considered feasible if the corresponding gap constraints hold across all $s_i \in S$. The VGLCSP represents a flexible and realistic model of sequence alignment, which is particularly useful for comparing DNA or protein sequences with variable structural distances, but also in time series analysis,

especially in settings where events are required to occur within specified temporal delays [8].

Although exact dynamic programming approaches exist for two-sequence case ($m = 2$) scenario [12], their computational complexity becomes prohibitive for larger m. To the best of our knowledge, no effective exact and only one approximate approaches for general m, a clearly NP-hard problem. The only existing approximate approach is an iterative beam search [1]. This methodology is built on the state-space problem representation and introduced space of root nodes. Each such node is explored in both ways via beam search: expanding current partial solution first by feasibly appending a sequence as its prefix, and subsequently, in another beam search call, feasibly extending that solution by concatenating letter-by-letter to its tail. At each iterations, new candidates for root nodes are detected and added in the candidate queue. Given that the methodology provides a scalable alternative to inefficient exact methods, this approach provided first results in the literature for arbitrary number of input sequences m. On the other hand, the approach relies on the use of several known hand-crafted (upper bound) heuristics for which there is an evidence from the LCS problem literature that they may offer poor guidance when m is large of Σ is small [10]. In the presence of gap constraints, it could be even worsen. The second detected research gap concerns lack of practically-motivated instances for the tackled problem as only synthetic instances are generated so far. Driven by these questions, the first gap is filled by designing a learning iterative beam search, which combines hand-crafted search guidance and the learned guidance into a superior one. To learn a search heuristic, an iterative approach is performed, able to learn the guidance in the iterative beam search approach. Shortly, the heuristic score for each node is represented as an outcome of multi-layered neural network derived in a layer-by-layer forward pass. The weights of neural network model are evolved by a genetic algorithm. The COP literature is known by utilizing similar (hybrid) methodology when learning heuristics, for example in [5] for the restricted LCS, [15] for the square LCS problem, or [17] for routing problems, to name a few. The difference between these is that their learned heuristics are not further combined with others as we did here into one ensemble method. To address the second research gap, we have derived practically-motivated problem instances by taking (two) biologically-motivated benchmark sets from LCS literature, and generating energy-aware gap constraints at each position of inherited input sequences.

Contributions. This work utilizes a framework for constructing a data-driven heuristic to guide the iterative beam search, specifically designed in the context of the *Variable Gapped Longest Common Subsequence Problem* (VGLCSP). A feed-forward neural network (FFNN) is used to parameterize the heuristic score of each node in the beam search. Instead of relying on gradient-based training, which is shown effective for the (constrained) continuous problems, the network weights are optimized using a Genetic Algorithm (GA), in particular the popular biased RKGA (BRKGA) approach. Note that here we deal with a fully non-differentiable search space when training this ML model. To establish syn-

ergy between two complementary scores, the outcome of the trained neural network and hand-crafted heuristics, a novel ensemble heuristic is introduced. This novel ensemble heuristic is clearly a new state-of-the-art guidance, according to the echausive experimental evaluation. The second contribution is the introduction of the first real-world benchmark set fo the tackled problem. In this case-scenario, gap constraints are designed to reflect on local thermodynamic stability estimated via nearest-neighbor interaction around each nucleotide. In this way, the purpose is to obtain a subsequence with stable molecular structure.

The paper is organized as follows. In Section 2, the iterative multi source beam search approach from the literature is roughly presented. Section 3 provides details on the process of learning heuristic guidance through multi-layer neural network, and the design of the combined heuristic estimator for the tacked problem. Section 4 reports a rigorous experimental analysis. Finally, Section 5 concludes the paper highlighting several future research directions.

2 Iterative Multi-Source Beam Search for GLCS Problem

The methodology of Iterative multi-source beam search (IMSBS) is proposed in [1] and here we only present it in some details. Interested readers are advised to check complete details in the above-cited paper.

This approach is derived from the general *state graph* formulation for the VGLCSP. Each node represents a partial feasible subsequence, characterized by the vector of current positions in each input sequence and the subsequence length. Specifically, a state $v = (\mathbf{p}^{L,v}, l^v)$ is induced by a partial solution s^v if: (i) $p_i^{L,v} - 1$ is the smallest index such that s^v is a subsequence of the prefix string $s_i[1] \cdots s_i[p_i^{L,v} - 1]$; (ii) $l^v = |s^v|$; and (iii) all the gaps G_{s_i} are preserved with s^v. A directed arc $\alpha = (v_1 v_2)$, labeled $lett(\alpha) = a \in \Sigma$, exists between states v_1 and v_2 if $l^{v_2} = l^{v_1} + 1$ and $s^{v_2} = s^{v_1} \cdot a$. To expand a state v, all letters that occur in each suffix of s_i defined by respective positions $p_i^{L,v}$ are considered. For each such letter a, the minimal positions $p_i^{L,a} \geq p_i^{L,v}$ of its appearances are identified fulfilling $p_i^{L,a} - p_i^{L,v} \leq G_{s_i}(p_i^{L,a})$. The resulting child node—if not suboptimal—is given by $w = (p^{L,a} + 1, l^v + 1)$. The root node $r := ((1, \ldots, 1), 0)$ represents the empty subsequence, while goal/target states correspond to non-extendable feasible subsequences.

Unlike the classical LCSP, the VGLCSP admits multiple (possibly exponentially many) root nodes as the starting position of a root node is constrained-free. For example, in sequences $s_1 =$ `ATGGAAAA` and $s_2 =$ `ATCCAAAA` with $G_{s_1} = G_{s_2} = 1$, the state $r = ((1, 1), 0)$ cannot reach any state with the position vector $(5, 5)$ via transitions, missing an opportunity to reach optimal subsequence `AAA` is starting the search from r. This motivates the development of IMSBS approach that works on a space of rooted nodes, which are dynamically generated over iterations and explored based on its priority. BS is a popular meta-heuristic approach [16] that performs in a breadth-first-search manner by expanding up to a certain number of nodes at the current level until the bottom is reached. This

amount of nodes is controlled by the size of beam (β)—which controls the level of greediness—and a heuristic guidance (h) for selecting them—as especially sensible algorithmic parameter.

In the IMSBS, BS is iteratively executed with a set of promising candidate root nodes as an initial beam, selected from a dynamically maintained global priority queue of candidate root nodes, controlling the diversification of the approach. Each iteration performs a complete BS on a few (num_root_nodes) top candidate root nodes, which are, beforehand, refined by the beam search with a small beam width β' and heuristic guidance h', as not all candidate nodes indeed represent root nodes. This process intensify the search and permit it from returning proven suboptimal solutions. During each BS call, new candidate root nodes are generated from the complete (non-expandable) nodes and subsequently added in the priority queue. This approach balances the intensification and diversification by several of its parameters. The stopping criteria for the IMSBS are the number of allowed beam search iterations (bs_iters), and time limit of 30 minutes.

In the remaining part of the paper, we refer to the algorithm as $\texttt{IMSBS}(\beta, h, \beta', h', num_root_nodes)$, returning an approximate solution s_{imsbs}.

3 Learning to Guide the Search

Neural networks are well-known machine learning model [6] that has shown its applicability in majority of fields [4]. There are many kind of neural networks such as the basic one, multi-layer perceptron, towards more complex Constitutional and recurrent neural networks of different architectures [11]. We will here keep the things simple, and will consider only the fully-connected (multi)layered neural networks that will serve as tool to guide the search process of our iterative beam-search approach instead of the hand-crafted versions used in [1]. For doing so, the major issue is to train the neural network, that is, setting its proper weights. In the context of classical neural networks for regression, this process is performed by alternating between forward and backward propagation, where the later is used to adapt the weights to minimize the error score on a set of training data w.r.t. a loss function. The process of weights adaptation is usually executed by employing iterative numerical procedures such as (stohastic) gradient descent [3] or versions thereof using the power of gradient in quickly reaching local optima and decreasing the overall loss w.r.t. known target values, hopping to generalize well on unseen data.

As we operate in combinatorial space of an NP-hard problem, we do not expect in general target values are known in advance, or to some extent these will be known but only for small-to-middle sized instances if an efficient exact solver exists. This limits the application of gradient descent algorithms in learning weights for such environments. Thus, we approach here differently, by employing meta-heuristic approach with a diverse population of solutions representing weights of NN, able to quickly evolve towards local optima. Therefore,

the back propagation pass in the classical NN training process is replaced by a population based meta-heuristic which adopts weights over iterations. In the feed forward pass fitness value for specific NN configuration has been determined.

Predefined Architecture of NN. The heuristic score is represented by a *multi-layer perceptron* with an number of hidden layers and user-defined units per layer, the standard input and output layer. The input layer received a set of numerical values associated to subproblem to be evaluated, and the output consists of one (real) value, indicating how the considered subproblem is "important". At each layer, to catch up non-linear relations between input features, we support the use of following activation functions: `hyperbolic tangent`, `ReLU`, and `sigmoid`.

For each layer l, forward propagation is defined as:

$$\mathbf{z}^{(l+1)} = W^{(l)}\mathbf{a}^{(l)} + b^{(l)}, \quad \mathbf{a}^{(l+1)} = \phi(\mathbf{z}^{(l+1)}),$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector, respectively, and ϕ denotes the activation function. All weights and biases are flattened into a **single vector**, which forms the chromosome of a GA individual.

Fitness Evaluation of NN configurations. Given a candidate weight vector w, we perform: 1) Load of the weights into a FFNN $\mathcal{N}(w)$ of fixed architecture; 2) Executing the IMSBS on each instance from a training set $\mathcal{T}$ utilizing the neural heuristic (to guide the search in forward iterations of IMSBS as a sensible step); 3) Recording size of the best sequence obtained for each instance.

The fitness value is computed as the average solution quality over all training instances. Formally, the learning problem is defined as:

$$\max_{w} \; \mathit{Fitness}(w) = \frac{1}{|T|} \sum_{I \in \mathcal{T}} f(\texttt{IMSBS}(I, h_w)), \tag{1}$$

where h_w is the heuristic induced by neural network $\mathcal{N}(w)$, and $f(\cdot)$ is a problem-specific objective function, in this case representing length of longest found solution in IMSBS guided by h_w. Validation performance score is computed analogously on a separate dataset $\mathcal{V}$ and is used only for reporting.

Feature Extraction for the VGLCSP. To guide the search process, each node is represented by a fixed-length numerical feature vector capturing both the current state in construction phase and the structure of induced subproblem to solve. These features are designed to be instance-independent.

Let $S = \{s_1, \ldots, s_k\}$ denote the set of input sequences, and let $v = (\mathbf{p}^v, l^v)$ be a given node. The vector $\mathbf{p}^L = (\mathbf{p}_1^v, \ldots, \mathbf{p}_m^v)$ is first normalized by the corresponding sequence lengths $F^v = (F^v)_i \leftarrow \frac{\mathbf{p}_i^v}{|s_i|}, i = 1, \ldots, m$ so that progress in each sequence lies in $[0, 1]$ and becomes independent of absolute input sizes.

From this normalized position vector, four statistical descriptors are extracted: maximum, minimum, mean, and standard deviation (4 features). These quantities characterize, respectively, the most advanced and least advanced sequences, the average depth of the search, and the imbalance of progress among

sequences. In addition, *the current length of partially constructed subsequence* l^v is included as a feature (the fifth feature). Together, these values provide a compact description of the current search state.

To capture the feasibility of future extensions, gap-related features are computed from the positions of remaining sequences. That is, for each v, we determine $G^{\min} = \min\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, $G^{\max} = \max\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, $G_i^{avg} = \text{mean}\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, and $G_i^{\text{std}} = \text{std}\{G_i(j) \mid j = \mathbf{p}_i^v, i = 1, \ldots, m\}$, where $G_1, \ldots, G_m$ are corresponding gap constraints associated to $s_1, \ldots, s_m$ respectively. This gives us additionally 4 features; in overall 9 features are comprised to create the **feature configuration 1** (F1). These statistics for gap constrains summarize the remaining flexibility of the instance: large average or maximum gaps indicate greater freedom for future matches, whereas small minimum gaps and large variability typically signal tighter constraints and increased difficulty. Additional feature $|\Sigma|$ creates the **feature configuration 2** (F2); When adding the value of m (number of input sequences) to F2, we obtain the **feature configuration 3** (F3). The alphabet size $|\Sigma|$ and the number of sequences (m) as the attributes provide global structural information about the instance and help the learning model adapt to varying problem scales. Note that we additionally probed with more features such as length of input sequences, but this has negative impact on solution quality of the learning approach.

Finally, all features are standardized to zero mean and unit variance prior to being passed to the learning component. The utilization of normalization is know to contribute decreased sensibility of learning process in the process of training ML models. Overall, the resulting representation combines information about (i) the current construction progress, (ii) the remaining gap-constraint landscape, and (iii) global instance properties, yielding between 9 and 11 features depending on the (3) configuration. This compact yet expressive encoding enables effective learning-based guidance of the search for high-quality subsequences.

Optimizer of NN Configurations. We opt for Genetic Algorithm, in particular its popular version called the Biased Random Key Genetic Algorithm (BRKGA) to solve the problem given in Eq. (1). The optimization process follows a population-based evolutionary scheme, described in Algorithm 1. Before we briefly explain the main workflow of the algorithm, first a few important notes.

Each individual is initialized by sampling weights uniformly from a predefined interval $[-w, w]$ (w is parametrized, but here fixed to 1). *Elitism* is included by copying a fixed number of top-performing individuals into the next generation. *Mutation* is performed so that several individuals are generated completely at random (random immigrants). This mechanism promotes exploration and maintains population diversity.

Concerning *offspring Generation* in BRKGA, one parent is selected from the elite set and one from the non-elite set. Each gene is inherited from the elite parent with a predefined probability (we fix it to 0.5).

The new population consists of elites, mutants, and offspring. The algorithm terminates once a predefined time limit is reached (in this case we used 2 hours). The best individual encountered during training is continuously tracked and stored. Once the new incumbent is reached, its validation score is also determined. If it is decreased in comparison to the best so far validation value, overfitting is detected and the learning process is immediately terminated.

Overall Learning Procedure. The learning process alternates between evolutionary optimization of network weights and evaluation through (iterative multi-source) Beam Search. Importantly, the neural network does not directly predict solutions; instead, it learns how to *guide* the search procedure. This places the method within the class of neuro-evolutionary hyper-heuristics.

Algorithm 1 GA-Based Learning of a BS Heuristic

Input: NN architecture $\mathcal{N}$, value of IMSBS parameters for *Fitness*, values of BRKGA parameters
Output: best weights for NN
Initialize population P with random neural network weights
Evaluate all individuals on training set $\mathcal{T}$
Store best solution w^*
$start \leftarrow$ current time
while time limit $t_{\max}$ has not exceeded **do**
  Sort P by $Fitness(\cdot)$ value (descending)
  Initialize empty population P'
  Copy top n_{elites} individuals from P to P'
  for $i = 1$ to $n_{mutants}$ **do**
    Create random individual x
    Evaluate x on $\mathcal{T}$, add x to P'
    Update w^* if improved & validation score not decreased
  end for
  $P \leftarrow P[: |P| - n_{mutants}]$
  for $i = 1$ to $n_{offspring}$ **do**
    Select parents from the elite set $P[: n_{elite}]$ and the rest of individuals $P[n_{elite} :]$
    Generate offspring by crossover based on the inherited probability p_{bias}
    Evaluate offspring on $\mathcal{T}$
    Add offspring to P'
    Update w^* if improved & validation score not decreased
  end for
  $P \leftarrow P'$
end while
return w^*

Given are multi-layer perception architecture $\mathcal{N}$, parameters of IMSBS, and parameters of BRKGA to guide the training process towards weights that induce high-quality heuristics.

The procedure starts by generating an initial population P of individuals, representing random weights to embed into the predefined neural network architecture. All individuals are evaluated on the training instance set $\mathcal{T}$, yielding a fitness value to reflect the quality of the corresponding heuristic. The best-performing individual found is stored as w^*.

Note that the fitness evaluation for each individual has been obtained by employing the state-of-the-art IMSBS approach for the VGLCSP on each instance from $\mathcal{T}$. However, time intensive IMSBS setting is not desirable as the training/learning process could take huge amount of time. We here opt for the following crucial parameters: $\beta = 500$, $num_beam_iters = 100$, while remaining parameters, therefore less sensible, are use from [1]. These values provide a good balance between the efficacy and the solution quality of the tuning process.

The main procedure of learning at each iteration executes the following steps. First, initial population of individuals P is sorted in descending order of fitness values. The worst $n_{mutants}$ individuals by to the fitness values are removed from P, and the same amount of individuals is randomly generated and added to the new population P' along with the n_{elite} elite individuals from P (which are copied to P'). The $n_{offspring}$ offsprings are generated by randomly selecting one parent from the pool of elite individuals, and other parent also randomly from the remaining individuals in P. These two are subsequently combined for generating new offspring which inherits the i-th component of first parent with probability $p_{bias} \geq 0.5$, otherwise the i-th component od second parent with the probability $1-p_{bias}$. The new offspring is passed to new generation of individuals P' after its fitness value is evaluated. For each such individual, its fitness score is calculated. In case a new incumbent has reached on the set of training instances $\mathcal{T}$, the fitness is calculated on the set of validation instances $\mathcal{V}$, assigning validation score to the individual. In particular, we decided to train in an early stopping fashion, terminating the training process whenever the validation value decreases, detecting the over-fitting. Otherwise, after generating all individuals, the population is updated as $P \leftarrow P'$. The process repeats until the time limit $t_{\max}$ is reached. The algorithm returns the best solution w^*, corresponding to the neural network weights that induce the most effective beam search heuristic on the training set.

As the best optimizer, the BRKGA setting used in training: $n_{pop} = 20, n_{elites} = 1, p_{bias} = 0.5, p_{mutants} = 7$.

3.1 An ensemble approach

To guide the (iterative) beam search, we employ a rank-based ensemble strategy that combines a learned heuristic with an analytical hand-crafted upper bound (UB) estimate for VGLCSP from [1]. Specifically, each candidate node in beam search is evaluated using two criteria: (i) a neural score obtained from a trained

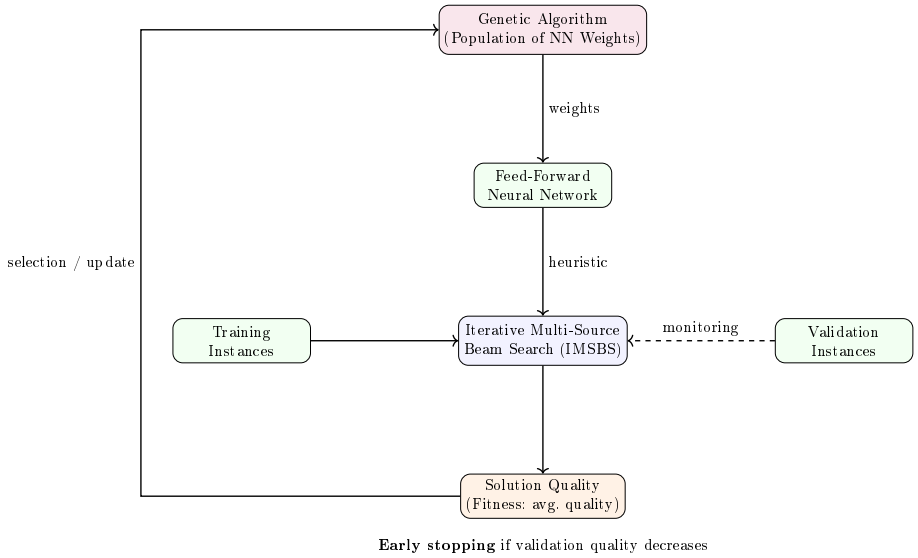

Fig. 1. Enhanced learning framework integrating a Genetic Algorithm with IMSBS. The GA evolves neural network weights, which define a heuristic used within the beam search. Training instances guide optimization, while validation instances monitor generalization. The feedback loop updates the population based on solution quality.

feed-forward neural network (FFNN), denoted $h_{w^*} = \mathcal{N}(w*)$, and (ii) a problem-specific upper bound (UB_2).

Instead of combining the raw scores directly, which may differ in scale and distribution, we adopt a rank aggregation approach. First, candidate nodes are sorted independently according to each criterion, producing two rankings. Each node is then assigned a rank based on its position in each sorted list.

The final ranking score is computed as a weighted combination of the two ranks:

$$r(v) = \alpha \cdot r_{h_{w^*}}(v) + (1 - \alpha) \cdot r_{\mathrm{UB}}(v),$$

where $\alpha = 0.45$ in our implementation except for $m = 3$ where $\alpha = 0.5$ is fixed. Nodes are then sorted in ascending order of $r(v)$, and the top candidates are retained according to the beam width.

This rank-based ensemble provides a robust mechanism for integrating learned and analytical guidance, mitigating scale inconsistencies and improving search stability.

4 Experimental Evaluation

In this section we compare the performance of the following methods are compared: (i) Imsbs: a baseline iterative beam search strategy from [1]; (ii) Limsbs:

Table 1. Comparison of objective values between IMSBS and LIMSBS on benchmark set RANDOM. Best results per instance category are displayed in bold.

m	n	$\|\Sigma\|$	LMSBS-ensemble	IMSBS
2	100	2	72.4	**72.9**
2	100	4	**62.0**	61.9
2	200	2	**142.5**	139.3
2	200	4	**125.9**	125.5
2	50	2	37.3	**37.7**
2	50	4	**30.1**	**30.1**
2	500	2	**305.9**	282.1
2	500	4	**305.5**	303.0
3	100	2	**58.5**	**58.5**
3	100	4	**48.7**	48.5
3	200	2	**106.7**	106.4
3	200	4	**97.9**	97.8
3	50	2	**31.2**	**31.2**
3	50	4	**22.9**	**22.9**
3	500	2	**154.9**	150.6
3	500	4	**237.6**	236.0
Avg.			**115.0**	112.8

m	n	$\|\Sigma\|$	LIMSBS-ensemble	IMSBS
5	100	2	32.6	**32.9**
5	100	4	28.0	**28.1**
5	200	2	**45.5**	42.1
5	200	4	**41.8**	38.8
5	50	2	**20.5**	**20.5**
5	50	4	**15.3**	**15.3**
5	500	2	**45.7**	39.2
5	500	4	**58.0**	45.6
10	100	2	**14.3**	12.0
10	100	4	**9.6**	8.9
10	200	2	**14.3**	12.8
10	200	4	**9.4**	8.2
10	50	2	**11.7**	**11.7**
10	50	4	7.3	**7.5**
10	500	2	**14.3**	13.2
10	500	4	**10.0**	8.6
Avg.			**23.6**	21.6

a IMSBS guided by the neural heuristic obtained in the process of learning; and (iii) IMSBS-ensemble: an IMSBS guided by the combined $r(\cdot)$ heuristic.

All approaches are implemented in C++ and compiled via GCC compiler, evaluated on the VEGA HPC system which is hosted at IZUM, Maribor. The cluster consists of 960 compute nodes equipped with AMD EPYC 7H12 CPUs operating at 2.35 GHz. All experiments were conducted in a single-threaded setting.

Benchmark instances. For the evaluation purposes, two type of benchmark sets are used. The first set of synthetic instances is proposed in [1], where for each combination of $n \in \{50, 100, 200, 500\}$, $m \in \{2, 3, 5, 10\}$, and $|\Sigma| \in \{2, 4\}$, 10 random problem instances are comprised. A total of *320 problem instances* is generated. The benchmark set is denoted as RANDOM. Apart from these, a set of biologically-motivated VGLCSP instances are here proposed. TODO: Marko: Explaining the way of generating the instances from benchmark set REAL!!!

All benchmark sets are freely available at XX.

Tuning process. IMSBS parameters are tuned by employing `irace` [14], producing the following IMSBS as the best configuration witin allowed training budget of 5000 runs (each run is limited to an execution time of 30 minutes): $\beta = 3000$, $h = h5$, $\#imbs_iters = 5000$.

All other (less sensible) parameters are set to the values reported in [1]. Concerning the learning IMSBS setting, it uses the same setting, to be completely fair with the comparison. For the training process, we experimented with different architectures for NN: 2-layer structures with 5 and 10 nodes per layer, and 3-layer configurations with 5 and 10 neurons.

Table 2. Comparison of LIMSBS and IMSBS on benchmark set Real.

RAT instances				
m	LIMSBS-ensemble	$\bar{t}[s]$	IMSBS	$\bar{t}[s]$
10	**28**	43.44	25	30.65
15	**11**	23.39	**11**	24.26
20	**10**	11.31	9	49.36
25	**9**	17.53	7	29.62
40	**5**	18.18	4	48.64
60	**3**	22.62	1	45.22
80	**1**	22.52	**1**	53.02
100	**1**	25.71	**1**	44.30
150	**1**	39.51	**1**	66.57
200	**1**	39.82	**1**	62.54

VIRUS instances				
m	LIMSBS-ensemble	$\bar{t}[s]$	IMSBS	$\bar{t}[s]$
10	**87**	134.51	36	43.14
15	**19**	27.08	**19**	36.52
20	**11**	16.31	10	26.92
25	**13**	16.66	9	30.40
40	**7**	20.39	5	32.12
60	**5**	20.26	4	34.38
80	**4**	22.80	3	40.64
100	**3**	35.27	**3**	43.25
150	**4**	27.71	3	31.80
200	**1**	32.06	**1**	48.59

In order to reduce time for tuning neural networks, we set up the same activation function in all layers, selecting among three (most) known: {`tanh`, `ReLU`, `sigmoid`}. After exhaustive experimental evaluations, We opt to chose the following configurations for the architecture of neural networks, one per different m on benchmark set Random:

- $m = 2$: units at each layer of FFNN: (10, 5, 5); activation function: `ReLu`; and the feature set (F2);
- $m = 3$: units at each layer of FFNN (10, 10, 5); activation function: `ReLu`; and the feature set (F2);
- $m = 5$: units at each layer of FFNN (10, 5, 5), activation function `sigmoid`; and the feature set (F1);
- $m = 10$: units at each layer of FFNN (10, 5, 5); activation function `ReLu`; and the feature set (F2).

The parameter $\alpha = 0.45$ in our ensemble approach except for $m = 3$ where $\alpha = 0.5$ as the results of grid search.

For the benchmark set Real, the architecture of neural network employed here is the one used for the benchmark set Random and the case of the largest $m = 10$ to verify the robustness of the learned heuristic also for different from random instances distributions with large m. (Note that here for all 20 instances m is larger than 10).

Numerical results on benchmark set Random. We evaluated the difference in solution quality between LIMSBS and IMSBS using the Wilcoxon signed-rank test on paired instances, reported in Table 3.

The test reveals a statistically significant difference in favor of LIMSBS, indicating that LIMSBS achieves superior performance overall. Out of 32 instances, LIMSBS outperforms IMSBS in 20 cases, while 7 instances result in ties and 5 favor IMSBS.

Numerical results on benchmark set Real. The following conclusions can be made form the obtained results, reprted in Table 2.

Table 3. Statistical comparison between LIMSBS and IMSBS using the Wilcoxon signed-rank test (one-sided).

Comparison	Wins / Ties / Losses	p-value	Conclusion
All instances ($m = 2, 3, 5, 10$)	20 / 7 / 5	0.009	LIMSBS significantly better
Small instances ($m = 2, 3$)	10 / 4 / 2	< 0.01	Significant difference
Large instances ($m = 5, 10$)	10 / 3 / 3	< 0.05	LIMSBS significantly better

- Overall, LIMSBS outperforms IMSBS in 15 out of 20 instances, while 5 instances result in ties and none favor IMSBS. This demonstrates a consistent dominance of LIMSBS across the tested dataset.
- In terms of solution quality, LIMSBS achieves an average value of 11.8, compared to 9.9 for IMSBS, corresponding to an average relative improvement of approximately 19%. The median quality values are 5 (LIMSBS) and 4 (IMSBS), confirming that improvements are consistent and not driven by outliers.
- The advantage of (ensemble) LIMSBS is particularly pronounced on more difficult instances (e.g., virus cases with higher variability), where substantial gains are observed (e.g., 87 vs. 36). In contrast, ties typically occur on simpler instances with very low objective values.
- Regarding computational time, IMSBS is generally faster, achieving an average runtime of 36.9 seconds compared to 32.6 seconds for the ensemble LIMSBS. However, this difference is not consistent across all instances, and in several cases LIMSBS is competitive or even faster.

The results again demonstrate that the ensemble LIMSBS provides a substantial and consistent improvement in solution quality over IMSBS on real-world instances, at the cost of a comparable computational effort. This makes LIMSBS a preferable choice when solution quality is the primary objective.

TODO: Marko: Came up to here!!!

5 Conclusions

Acknowledgments. This publication is co- funded by the European Union's Horizon Europe research and innovation program under the Marie Sklodowska-Curie COFUND Postdoctoral Programme grant agreement No.101081355– SMASH and by the Republic of Slovenia and the European Union from the European Regional Development Fund. Co-funded by European Union. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or European Research Executive Agency (REA). Neither the European Union nor the REA can be held responsible for them.

References

1.

2. Adamson, D., Kosche, M., Koß, T., Manea, F., Siemer, S.: Longest common subsequence with gap constraints. In: International Conference on Combinatorics on Words. pp. 60–76. Springer (2023)
3. Amari, S.i.: Backpropagation and stochastic gradient descent method. Neurocomputing **5**(4-5), 185–196 (1993)
4. Bishop, C.M.: Neural networks and their applications. Review of scientific instruments **65**(6), 1803–1832 (1994)
5. Djukanović, M., Reixach, J., Nikolikj, A., Eftimov, T., Kartelj, A., Blum, C.: A learning search algorithm for the restricted longest common subsequence problem. Expert Systems with Applications **284**, 127731 (2025)
6. Gurney, K.: An introduction to neural networks. CRC press (2018)
7. Lafond, M., Lai, W., Liyanage, A., Zhu, B.: The longest subsequence-repeated subsequence problem. In: International Conference on Combinatorial Optimization and Applications. pp. 446–458. Springer (2023)
8. Lainscsek, C., Sejnowski, T.J.: Delay differential analysis of time series. Neural computation **27**(3), 594–614 (2015)
9. Lin, G., Chen, Z.Z., Jiang, T., Wen, J.: The longest common subsequence problem for sequences with nested arc annotations. Journal of Computer and System Sciences **65**(3), 465–480 (2002)
10. M. Djukanovic, G.R., Blum, C.: Finding longest common subsequences: New anytime A* search results. Applied Soft Computing **95**, 106499 (2020)
11. Mienye, I.D., Swart, T.G., Obaido, G.: Recurrent neural networks: A comprehensive review of architectures, variants, and applications. Information **15**(9), 517 (2024)
12. Peng, Y.H., Yang, C.B.: Finding the gapped longest common subsequence by incremental suffix maximum queries. Information and Computation **237**, 95–100 (2014)
13. Penga, Y.H., Yangb, C.B.: The longest common subsequence problem with variable gapped constraints. In: Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan. pp. 17–23 (2011)
14. Pérez Cáceres, L., López-Ibáñez, M., Stützle, T.: An analysis of parameters of irace. In: European Conference on Evolutionary Computation in Combinatorial Optimization. pp. 37–48. Springer (2014)
15. Reixach, J., Blum, C., Djukanović, M., Raidl, G.R.: A neural network based guidance for a brkga: An application to the longest common square subsequence problem. In: European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar). pp. 1–15. Springer (2024)
16. Wiseman, S., Rush, A.M.: Sequence-to-sequence learning as beam-search optimization. In: Proceedings of the 2016 conference on empirical methods in natural language processing. pp. 1296–1306 (2016)
17. Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A.: Learning improvement heuristics for solving routing problems. IEEE transactions on neural networks and learning systems **33**(9), 5057–5069 (2021)